

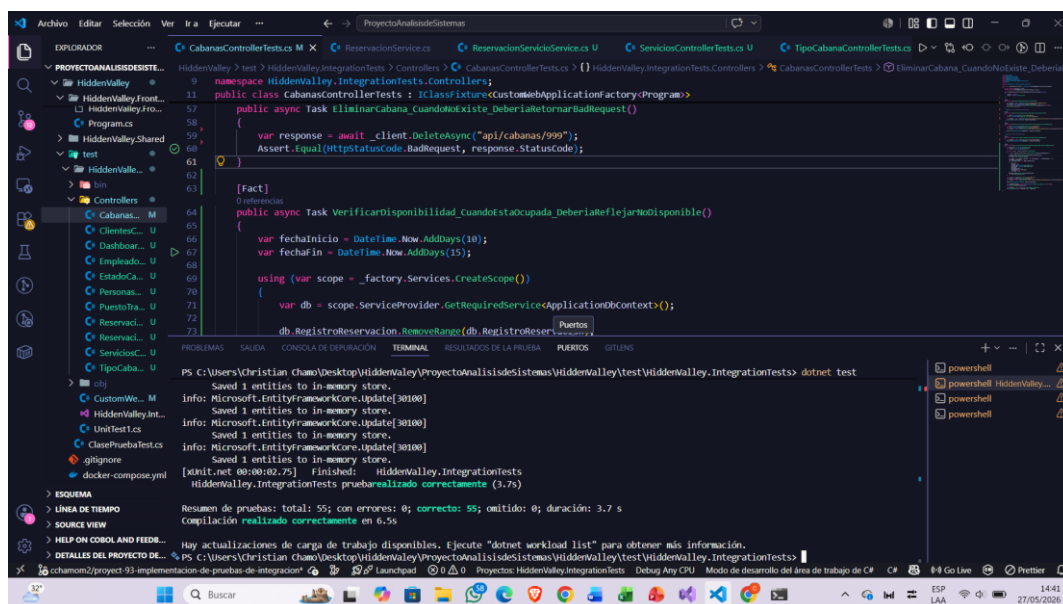
PRUEBAS DE INTEGRACION

Las pruebas de integración verifican que diferentes módulos o componentes de tu aplicación funcionen correctamente al interactuar entre sí. A diferencia de las pruebas unitarias que evalúan una clase aislada, las pruebas de integración validan flujos completos, incluyendo conexiones a bases de datos, APIs y sistemas de archivos

Confirman que tu código interactúa correctamente con dependencias reales (como PostgreSQL) en lugar de usar datos simulados (mocks). Identifican problemas de comunicación entre capas de tu aplicación (por ejemplo, el controlador llamando al repositorio) antes de llegar a producción.

Para implementar estas pruebas en el ecosistema de .NET, utilizamos librerías de ejecución de pruebas con utilidades específicas para levantar el entorno web:

- **Frameworks de pruebas:** xUnit, que se encargan de ejecutar los casos de prueba y evaluar los resultados.
- **Microsoft.AspNetCore.Mvc.Testing:** Una herramienta oficial que permite levantar un servidor web en memoria (TestServer) para realizar peticiones HTTP reales a tu API durante la prueba.



ISSUE LINEAR: <https://linear.app/hospital-veterinario-patitas/issue/PROYECTO-93/implementacion-de-pruebas-de-integracion>

Git Hub de la rama: <https://github.com/lcarranzag2-ui/ProyectoAnálisisdeSistemas/tree/cchamom2/proyecto-93-implementacion-de-pruebas-de-integracion>

CABAÑAS

The screenshot shows the Visual Studio IDE with the 'CabanasControllerTests.cs' file open. The test 'VerificarDisponibilidad_FechasValidas_DeberiaRetornarOk' has passed. The test run summary indicates 1 test passed, 0 failed, and 0 skipped, with a total execution time of 0.47s (470ms). The test results pane on the right shows 11 previous results.

```
namespace HiddenValley.IntegrationTests.Controllers;
public class CabanasControllerTests : IClassFixture<CustomWebApplicationFactory<Program>>
{
    [Fact]
    public async Task VerificarDisponibilidad_FechasValidas_DeberiaRetornarOk()
    {
        var inicio = DateTime.Now.ToString("yyyy-MM-dd");
        var fin = DateTime.Now.AddDays(3).ToString("yyyy-MM-dd");
        var response = await _client.GetAsync($"api/cabanas/disponibilidad?fechaInicio={inicio}&fechaFin={fin}");
        Assert.Equal(HttpStatusCode.OK, response.StatusCode);
    }

    [Fact]
    public async Task VerificarDisponibilidad_FechasInvertidas_DeberiaRetornarBadRequest()
    {
        var inicio = DateTime.Now.AddDays(5).ToString("yyyy-MM-dd");
        var fin = DateTime.Now.AddDays(2).ToString("yyyy-MM-dd");
    }
}
```

Test Run Summary

Total Tests:	1
Passed:	1
Failed:	0
Skipped:	0
Result:	SUCCEEDED
Test Execution Time:	0.47s (470ms)
Total Time (including build):	3.44s (3436ms)

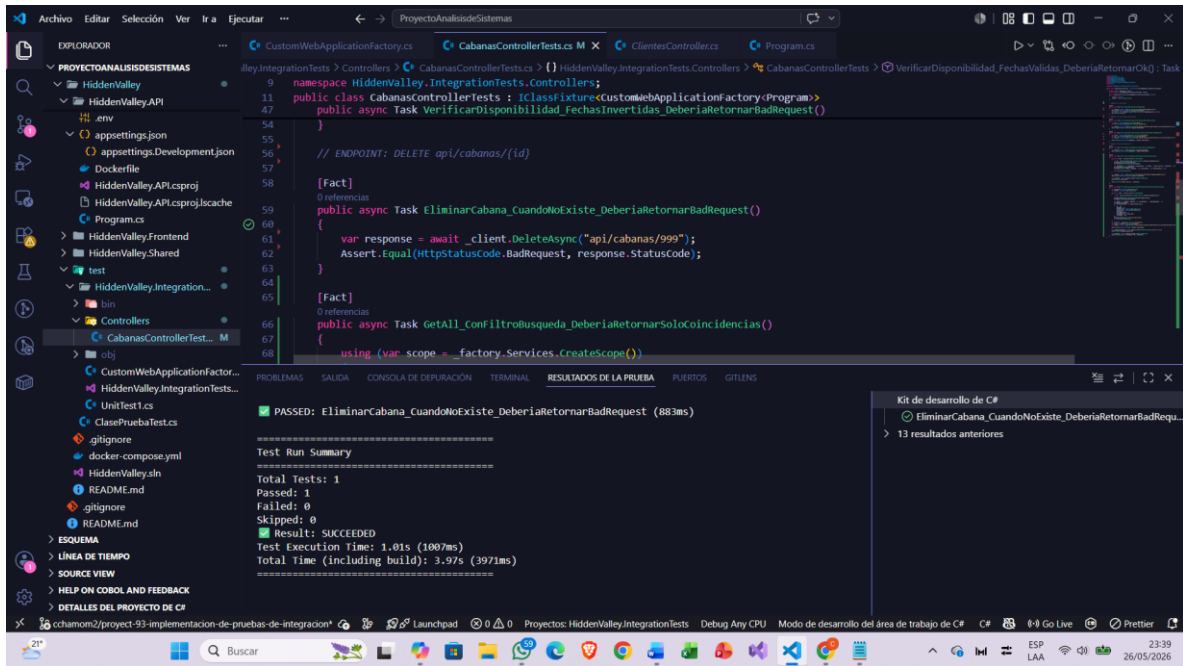
The screenshot shows the Visual Studio IDE with the 'CabanasControllerTests.cs' file open. The test 'VerificarDisponibilidad_FechasValidas_DeberiaRetornarOk' has passed. The test run summary indicates 1 test passed, 0 failed, and 0 skipped, with a total execution time of 1.04s (1036ms). The test results pane on the right shows 12 previous results.

```
namespace HiddenValley.IntegrationTests.Controllers;
public class CabanasControllerTests : IClassFixture<CustomWebApplicationFactory<Program>>
{
    [Fact]
    public async Task VerificarDisponibilidad_FechasValidas_DeberiaRetornarOk()
    {
        var inicio = DateTime.Now.ToString("yyyy-MM-dd");
        var fin = DateTime.Now.AddDays(3).ToString("yyyy-MM-dd");
        var response = await _client.GetAsync($"api/cabanas/disponibilidad?fechaInicio={inicio}&fechaFin={fin}");
        Assert.Equal(HttpStatusCode.OK, response.StatusCode);
    }

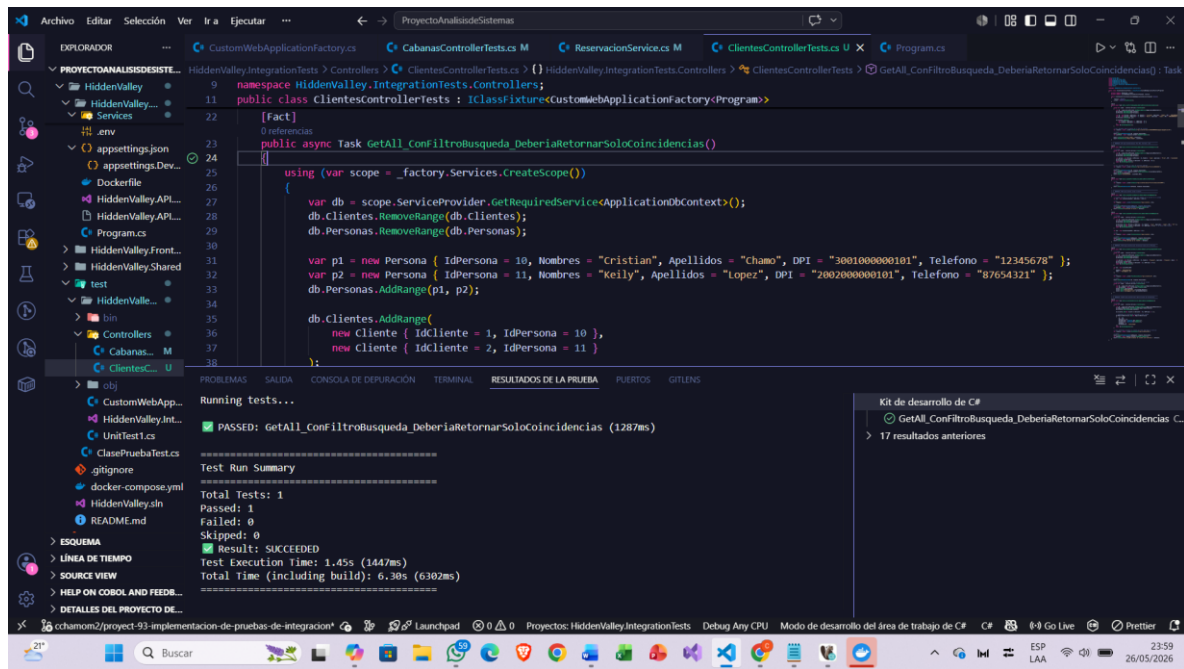
    [Fact]
    public async Task VerificarDisponibilidad_FechasInvertidas_DeberiaRetornarBadRequest()
    {
        var inicio = DateTime.Now.AddDays(5).ToString("yyyy-MM-dd");
        var fin = DateTime.Now.AddDays(2).ToString("yyyy-MM-dd");
    }
}
```

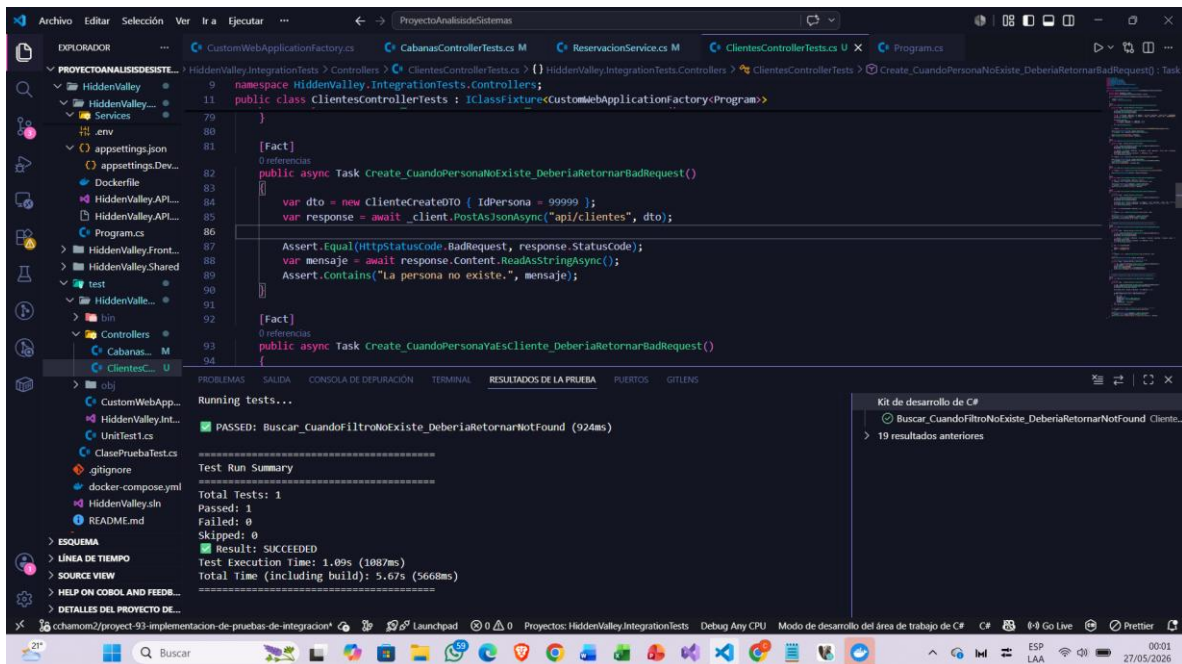
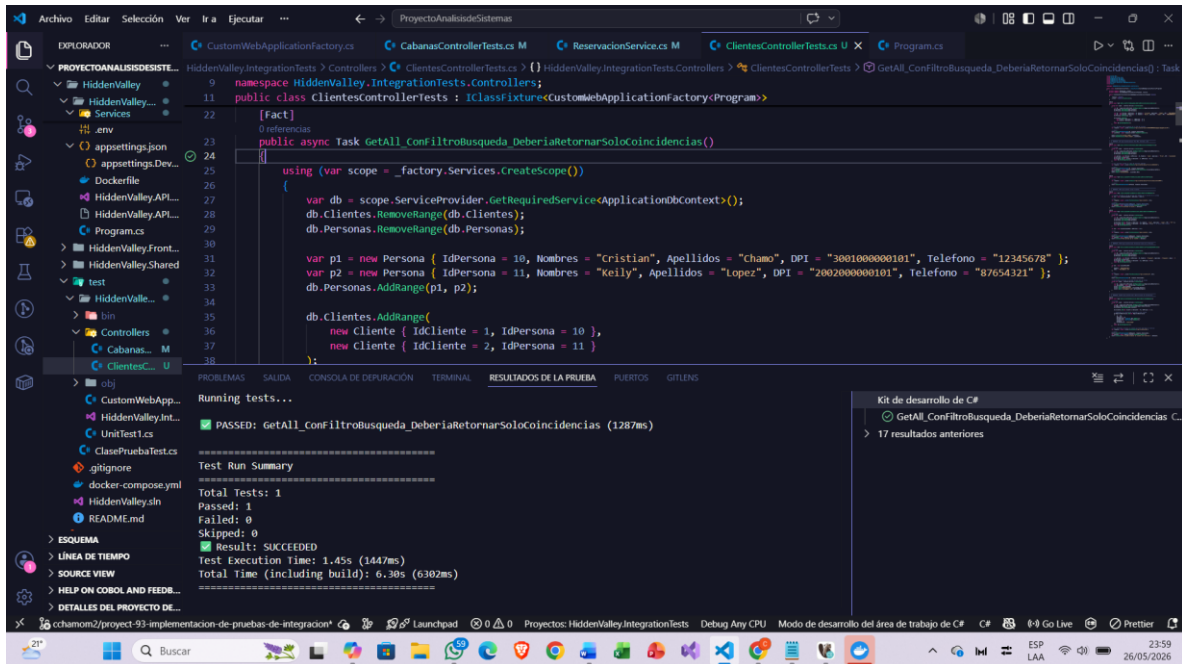
Test Run Summary

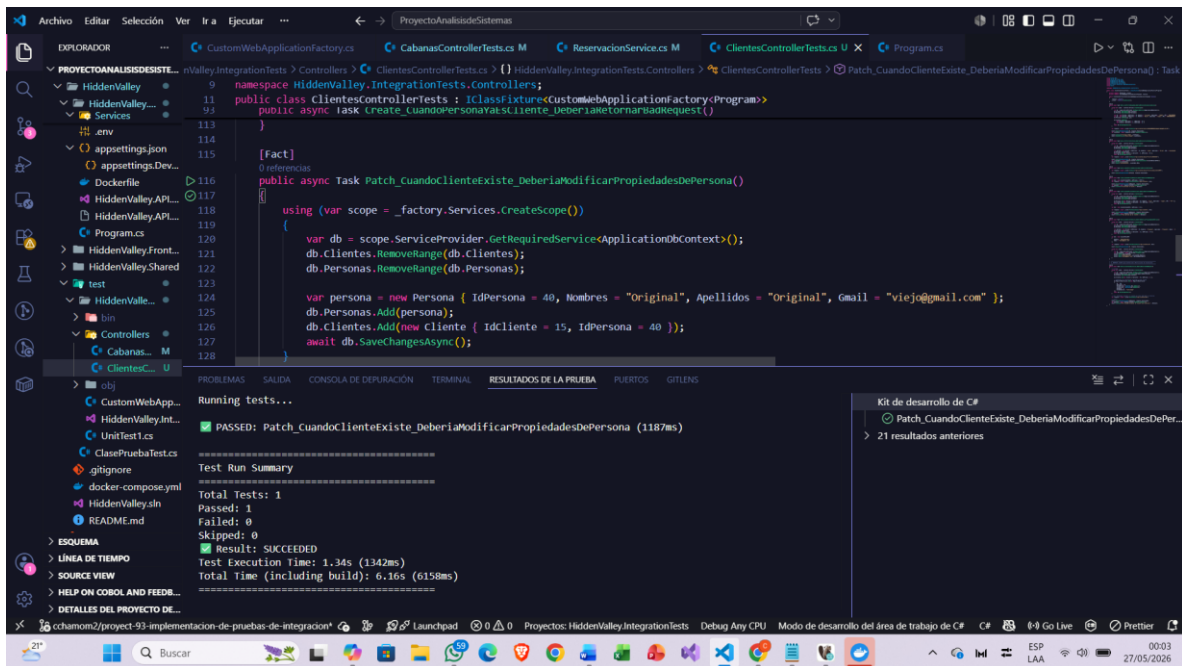
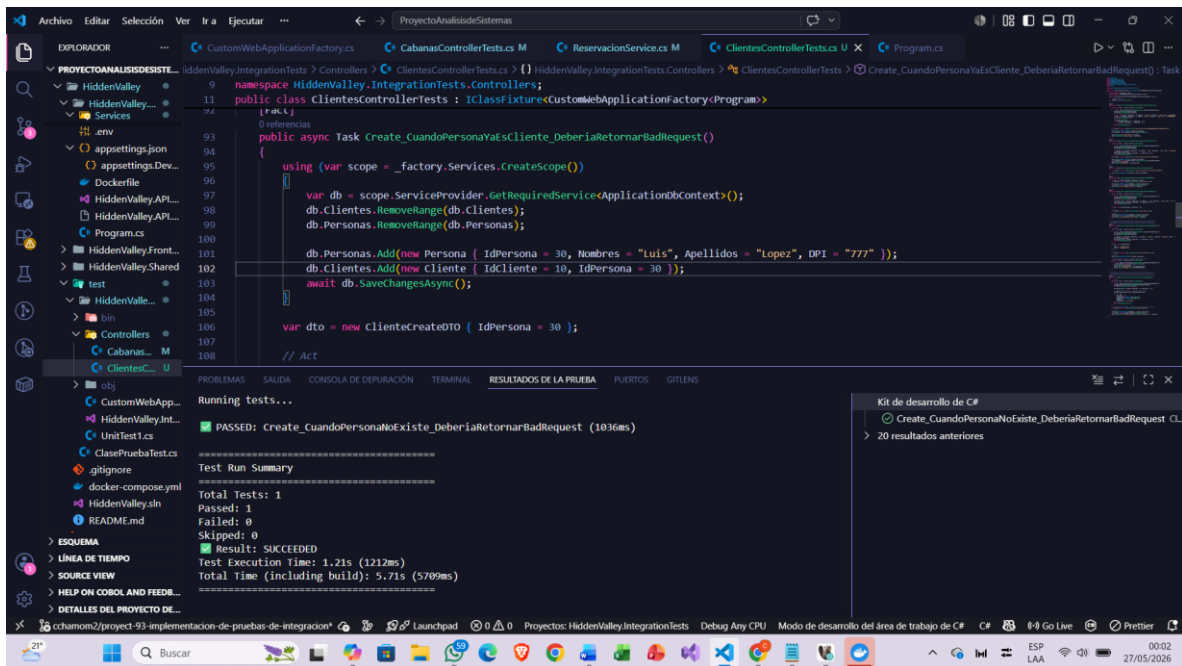
Total Tests:	1
Passed:	1
Failed:	0
Skipped:	0
Result:	SUCCEEDED
Test Execution Time:	1.04s (1036ms)
Total Time (including build):	3.97s (3965ms)

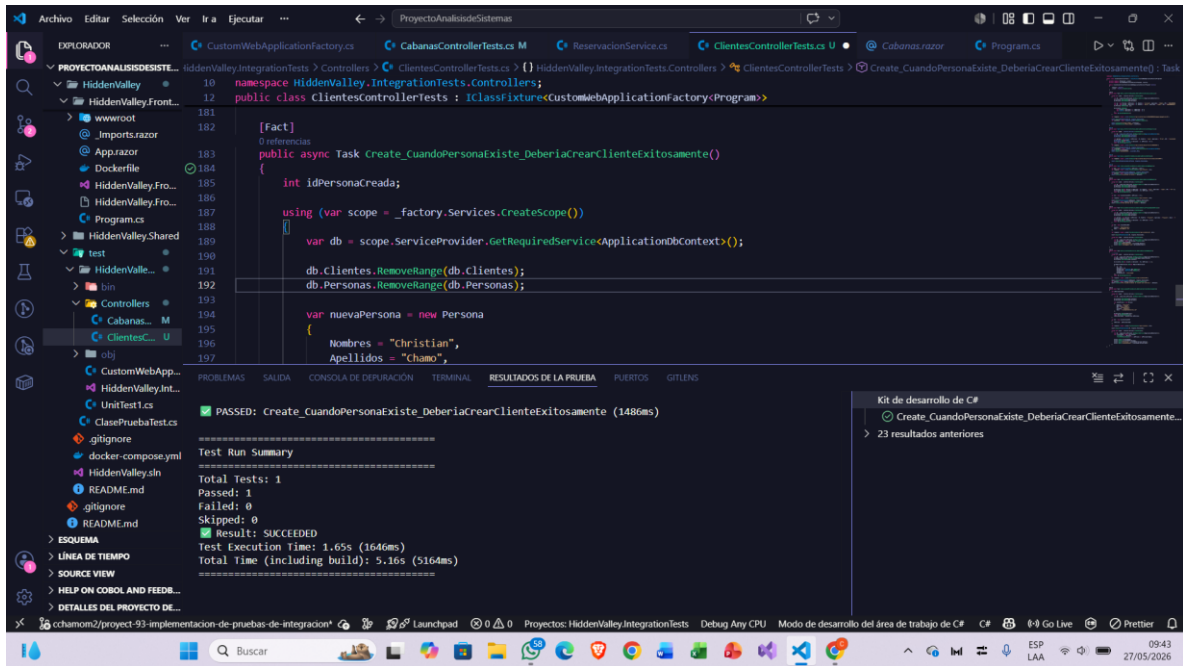


CLIENTES

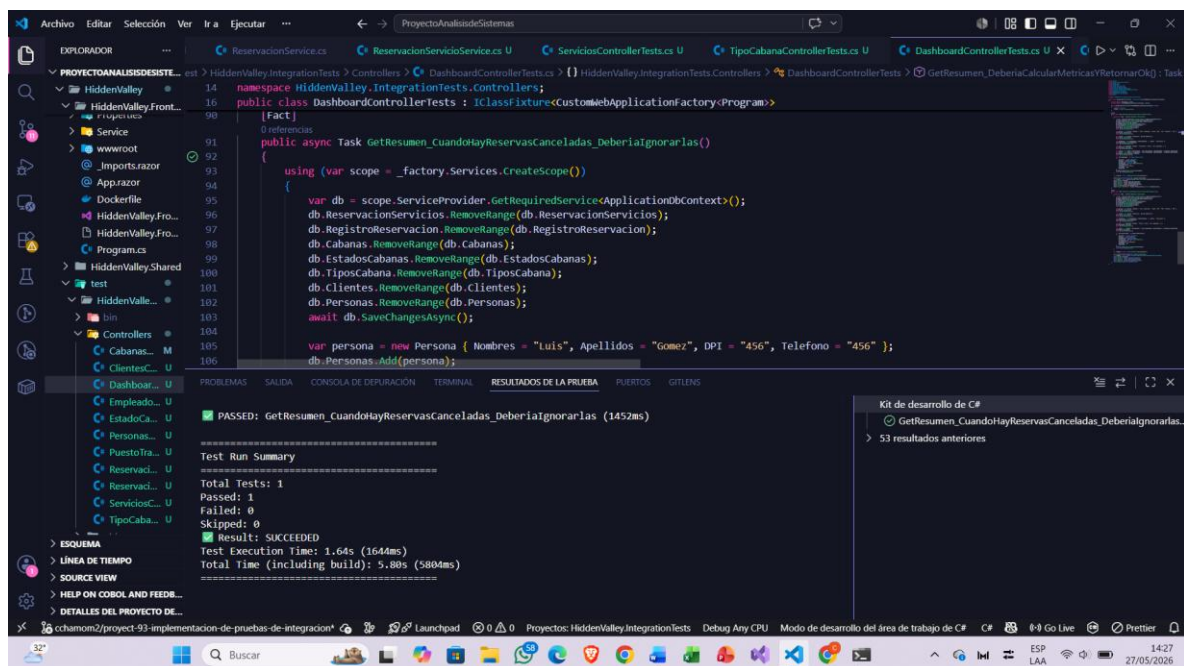
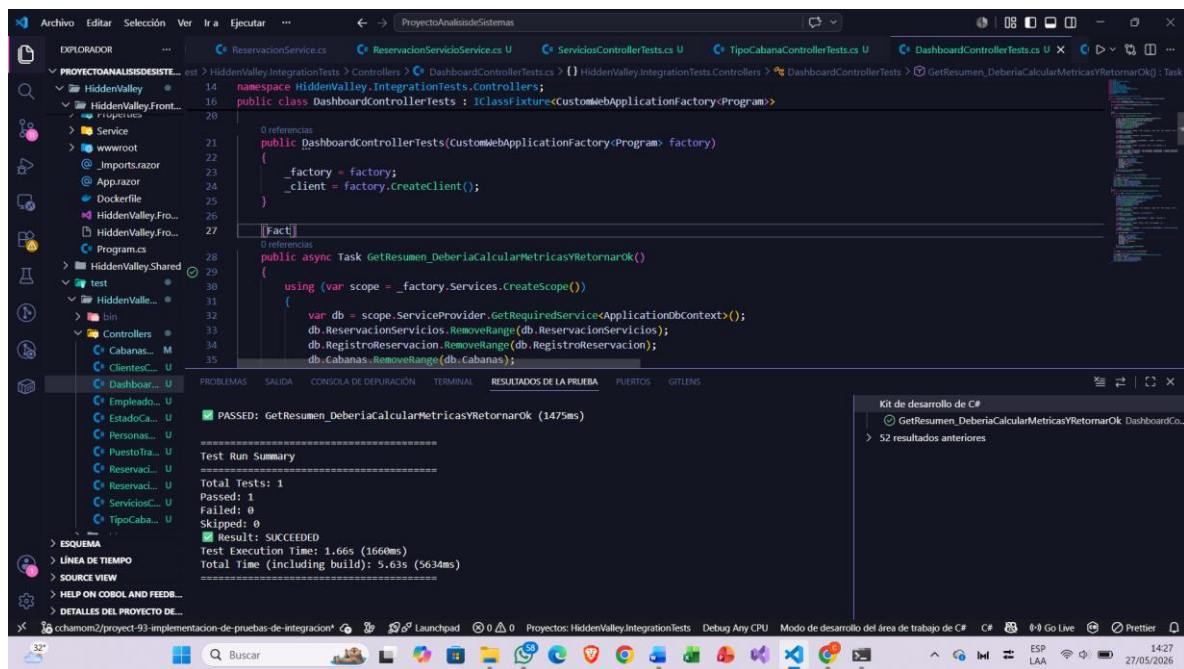




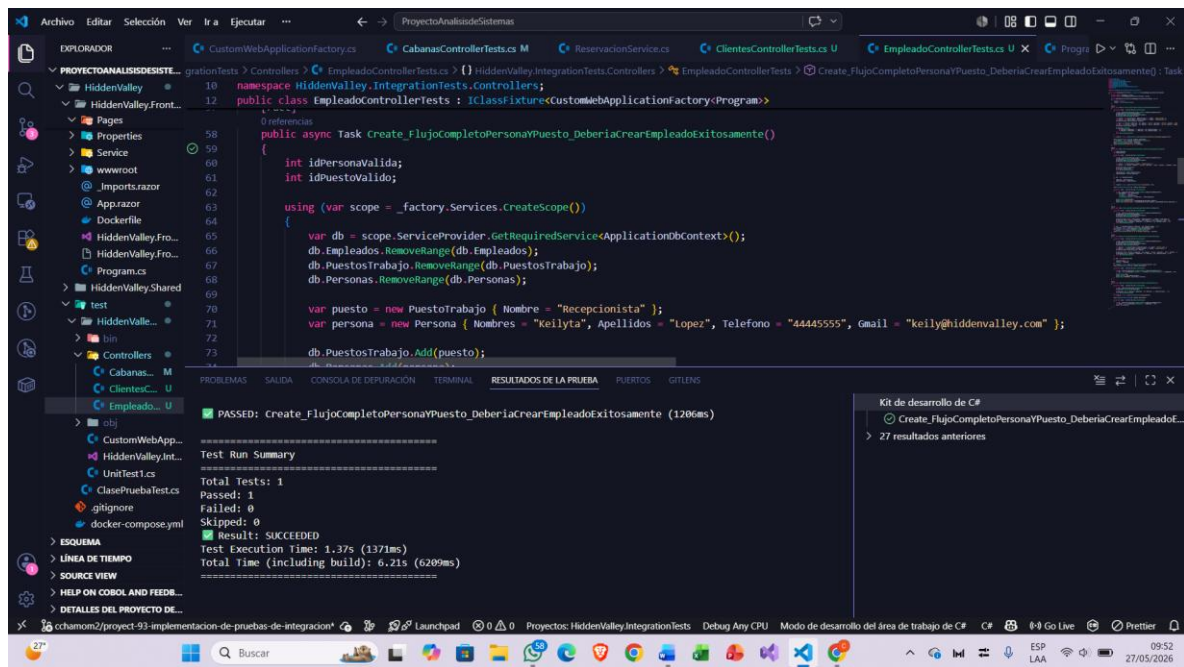
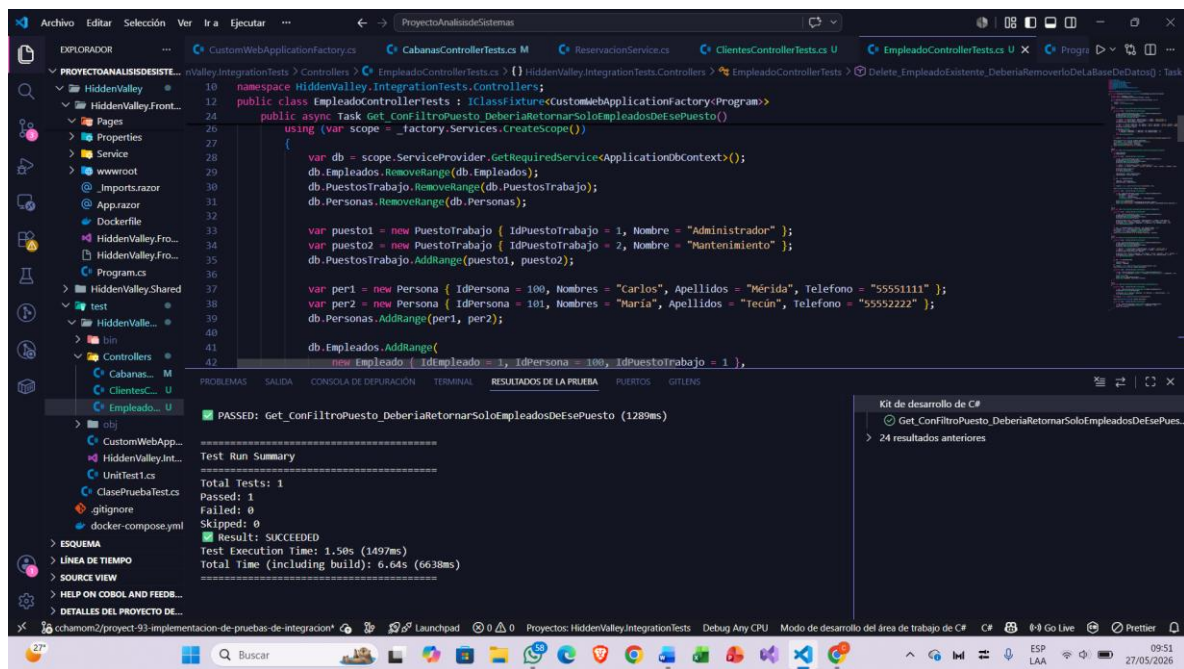


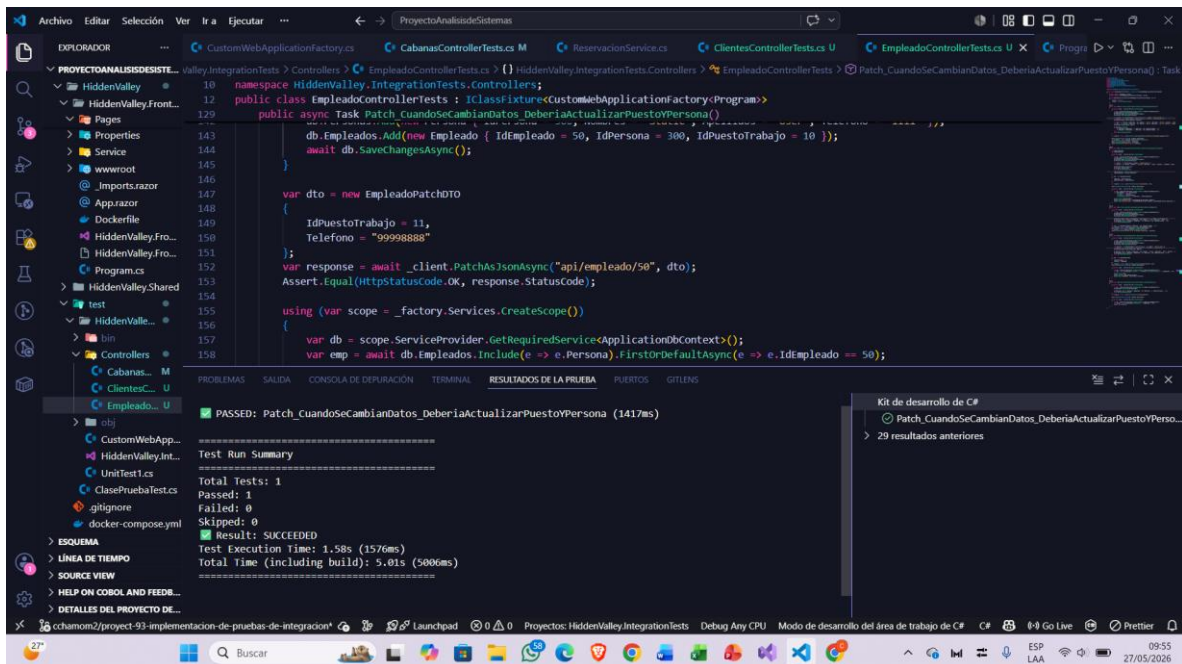
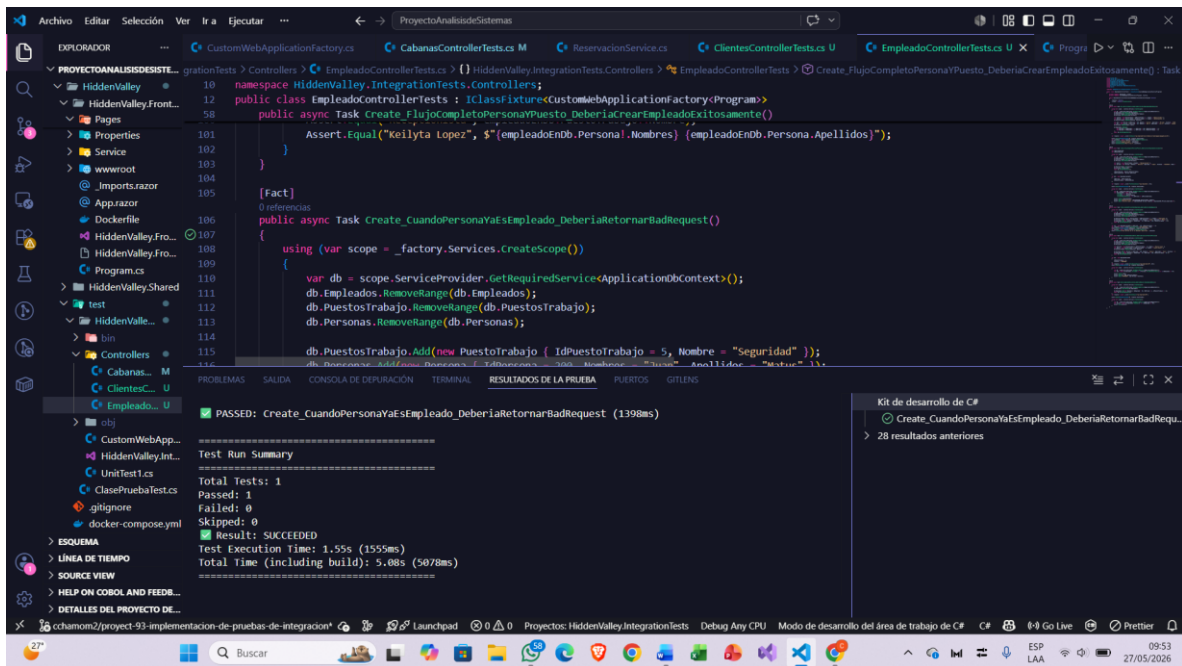


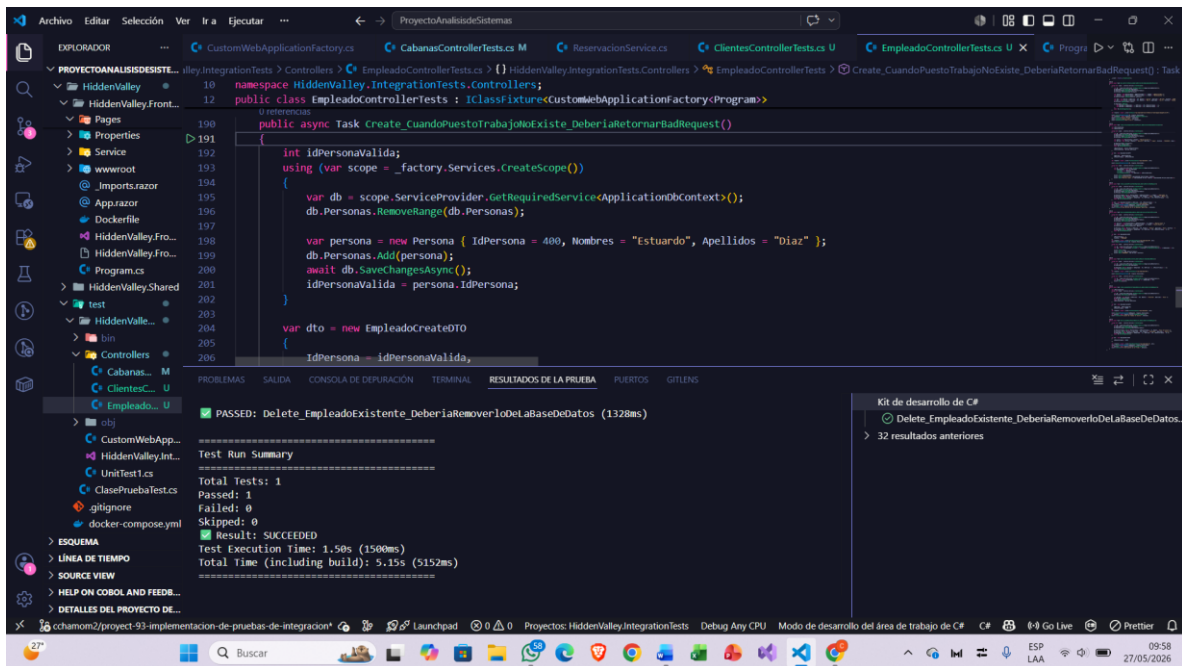
DASHBOARD



EMPLEADO







ESTADO CABAÑA

The screenshot shows the Visual Studio IDE with the 'EstadoCabanaControllerTests.cs' file open. The test method 'GetAll_DeberiaRetornarTodosLosEstadosDeLaBaseDeDatos' is highlighted. The test runner shows a 'PASSED' status with a duration of 1060ms. The test run summary indicates 1 total test, 1 passed, 0 failed, and 0 skipped. The test execution time is 1.22s (1222ms) and the total time including build is 6.01s (6015ms).

```
namespace HiddenValley.IntegrationTests.Controllers;
public class EstadoCabanaControllerTests : IClassFixture<CustomWebApplicationFactory<Program>>
{
    [Fact]
    public async Task GetAll_DeberiaRetornarTodosLosEstadosDeLaBaseDeDatos()
    {
        // Arrange
        using (var scope = _factory.Services.CreateScope())
        {
            var db = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();
            db.EstadosCabanas.RemoveRange(db.EstadosCabanas);

            db.EstadosCabanas.AddRange(
                new EstadoCabana { IdEstadoCabana = 1, Nombre = "Disponible", Descripcion = "Lista para usarse" },
                new EstadoCabana { IdEstadoCabana = 2, Nombre = "Mantenimiento", Descripcion = "En limpieza profunda" }
            );
            await db.SaveChangesAsync();
        }

        var response = await _client.GetAsync("api/estadocabana");
    }
}
```

Running tests...
PASSED: GetAll_DeberiaRetornarTodosLosEstadosDeLaBaseDeDatos (1060ms)

Test Run Summary
Total Tests: 1
Passed: 1
Failed: 0
Skipped: 0
Result: SUCCEEDED
Test Execution Time: 1.22s (1222ms)
Total Time (including build): 6.01s (6015ms)

The screenshot shows the Visual Studio IDE with the 'EstadoCabanaControllerTests.cs' file open. The test method 'GetById_CuandoExiste_DeberiaRetornarEstadoEspecifico' is highlighted. The test runner shows a 'PASSED' status with a duration of 1045ms. The test run summary indicates 1 total test, 1 passed, 0 failed, and 0 skipped. The test execution time is 1.21s (1206ms) and the total time including build is 5.80s (5800ms).

```
namespace HiddenValley.IntegrationTests.Controllers;
public class EstadoCabanaControllerTests : IClassFixture<CustomWebApplicationFactory<Program>>
{
    public async Task GetById_CuandoExiste_DeberiaRetornarEstadoEspecifico()
    {
        var db = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();
        db.EstadosCabanas.RemoveRange(db.EstadosCabanas);

        db.EstadosCabanas.Add(new EstadoCabana { IdEstadoCabana = 5, Nombre = "Ocupada", Descripcion = "Huéspedes activos" });
        await db.SaveChangesAsync();
    }

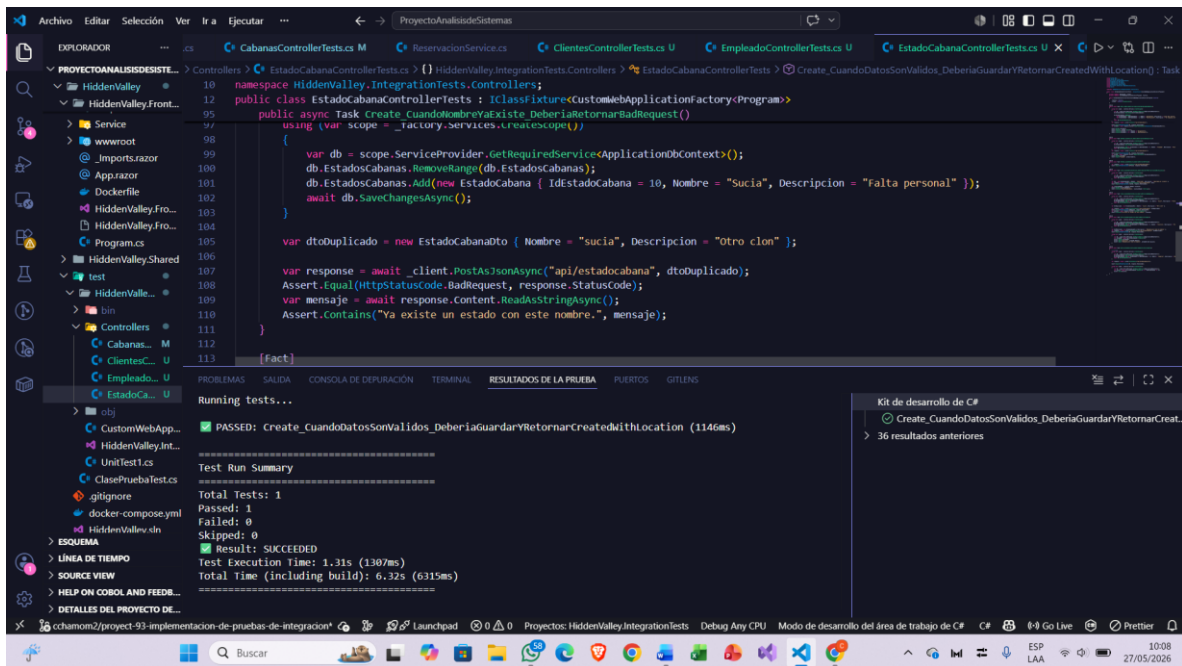
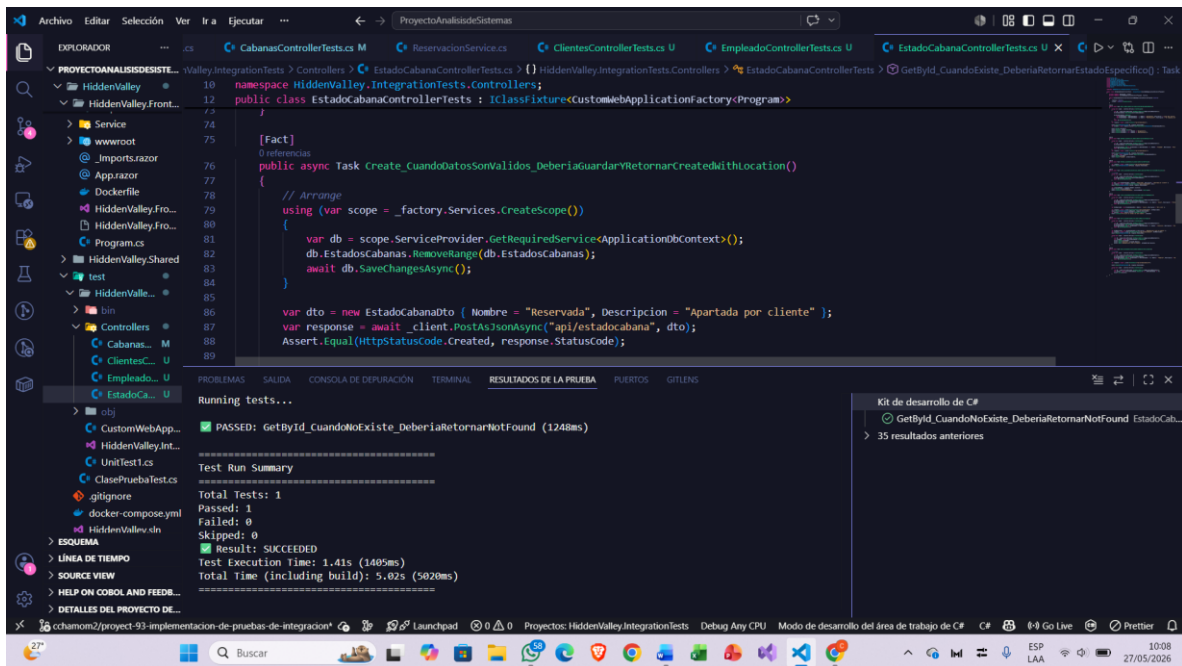
    var response = await _client.GetAsync("api/estadocabana/5");

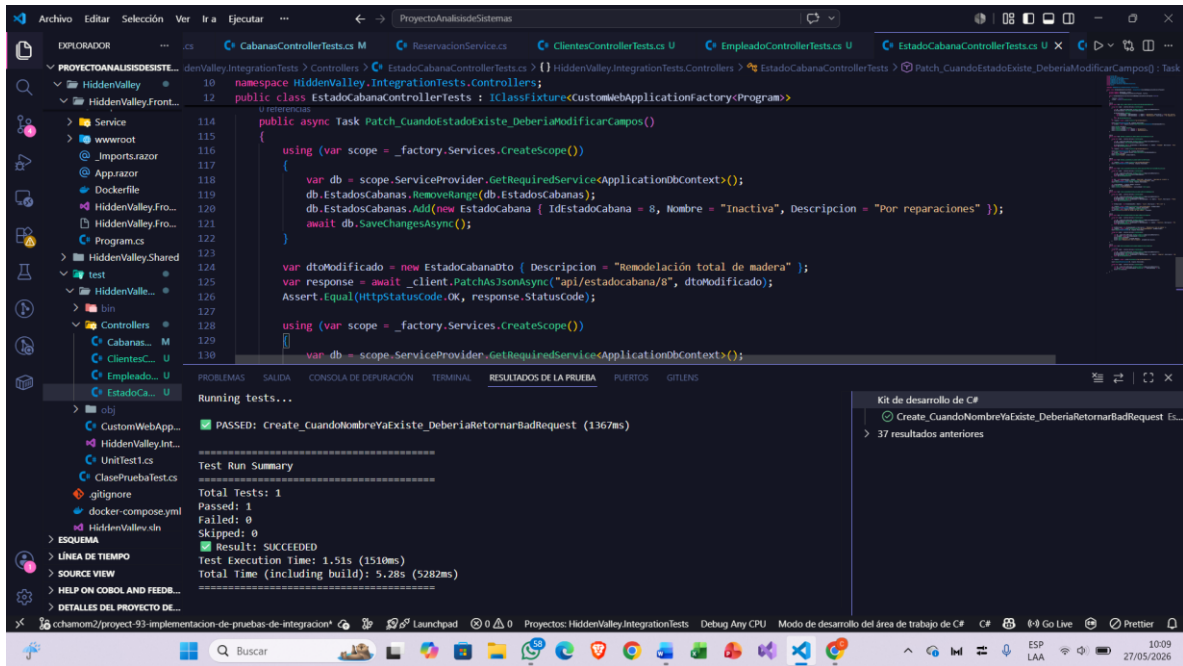
    Assert.Equal(HttpStatusCode.OK, response.StatusCode);
    var estado = await response.Content.ReadFromJsonAsync<EstadoCabanaDto>();
    Assert.NotNull(estado);
    Assert.Equal("Ocupada", estado.Nombre);
}

[Fact]
```

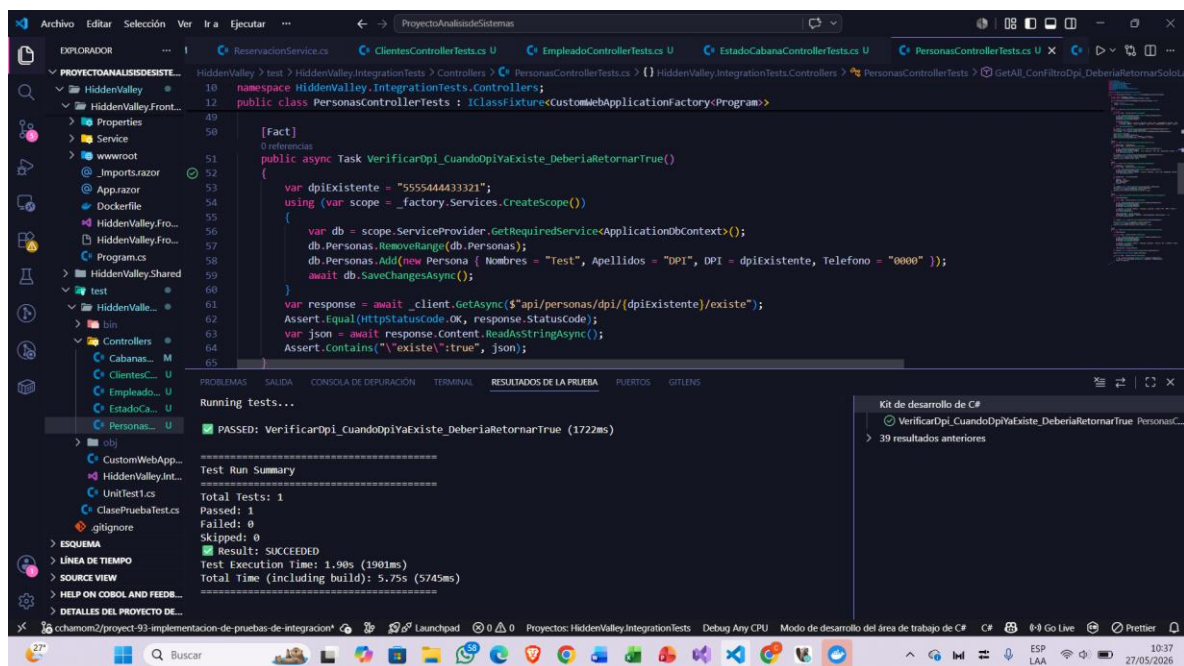
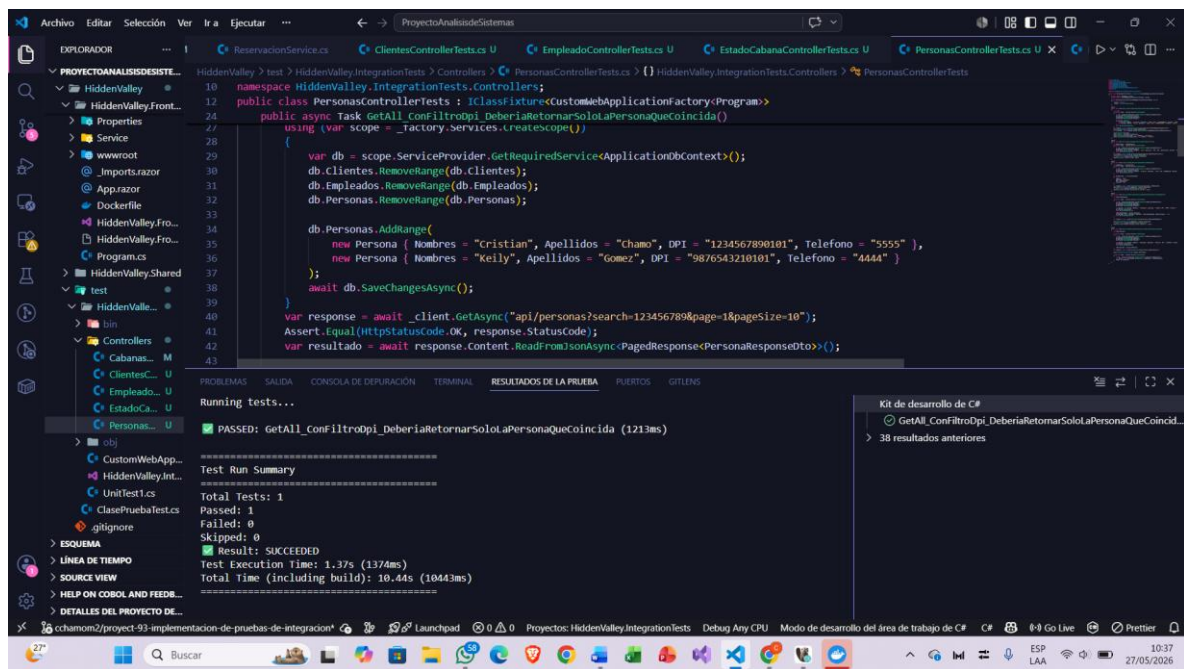
Running tests...
PASSED: GetById_CuandoExiste_DeberiaRetornarEstadoEspecifico (1045ms)

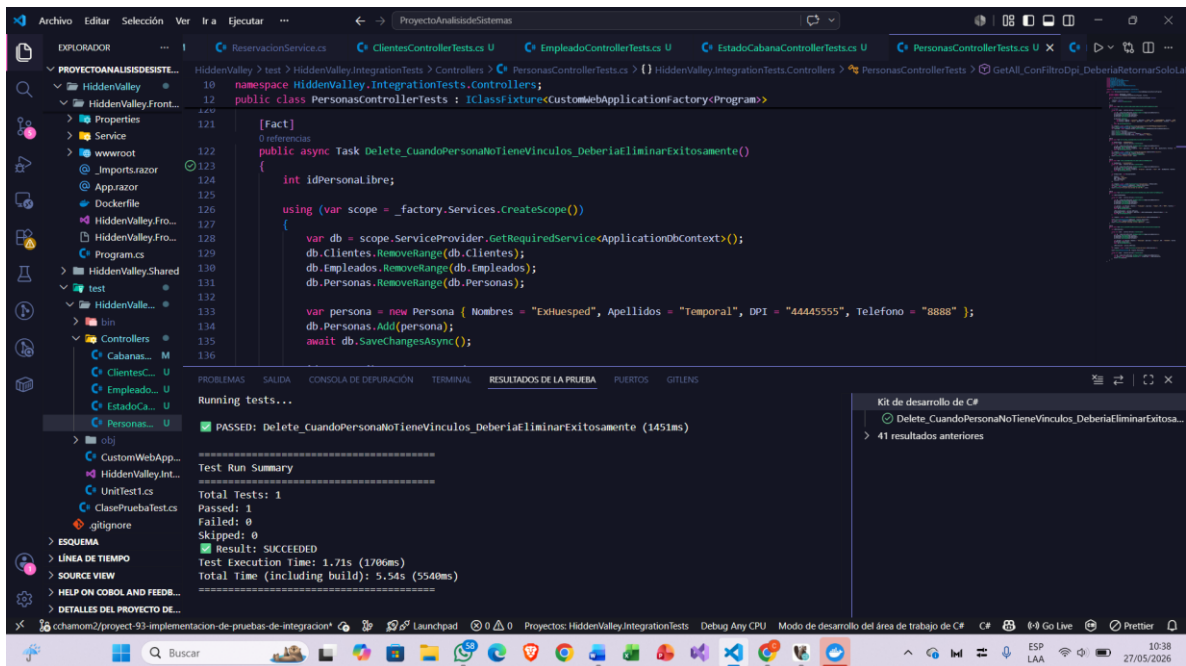
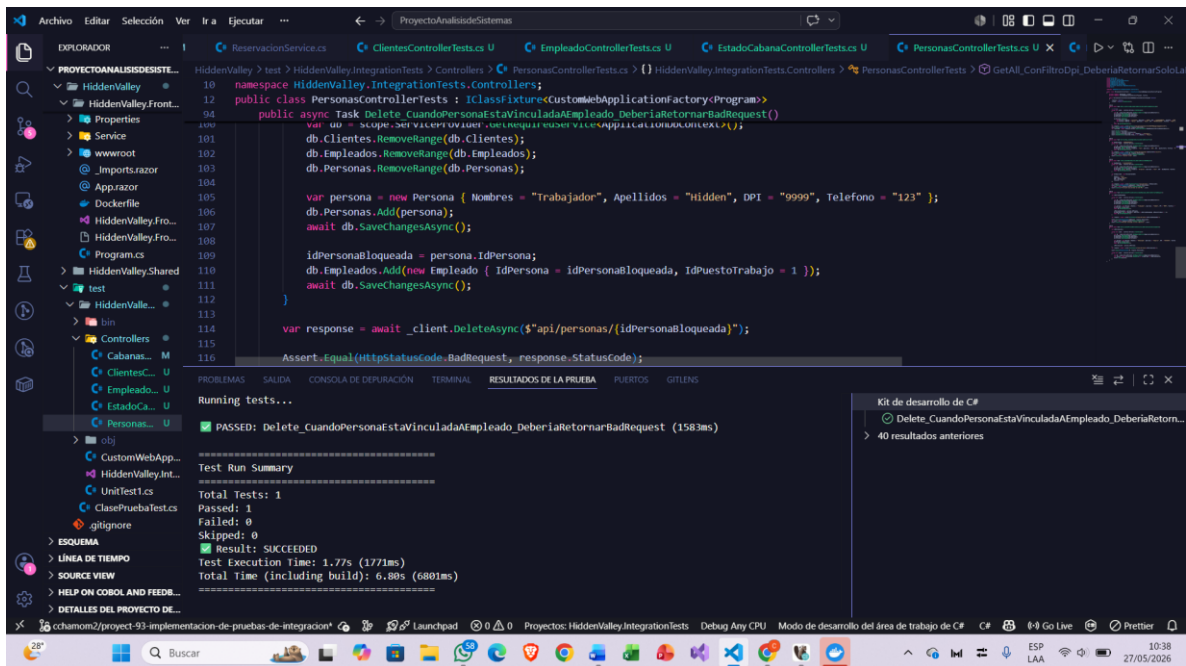
Test Run Summary
Total Tests: 1
Passed: 1
Failed: 0
Skipped: 0
Result: SUCCEEDED
Test Execution Time: 1.21s (1206ms)
Total Time (including build): 5.80s (5800ms)

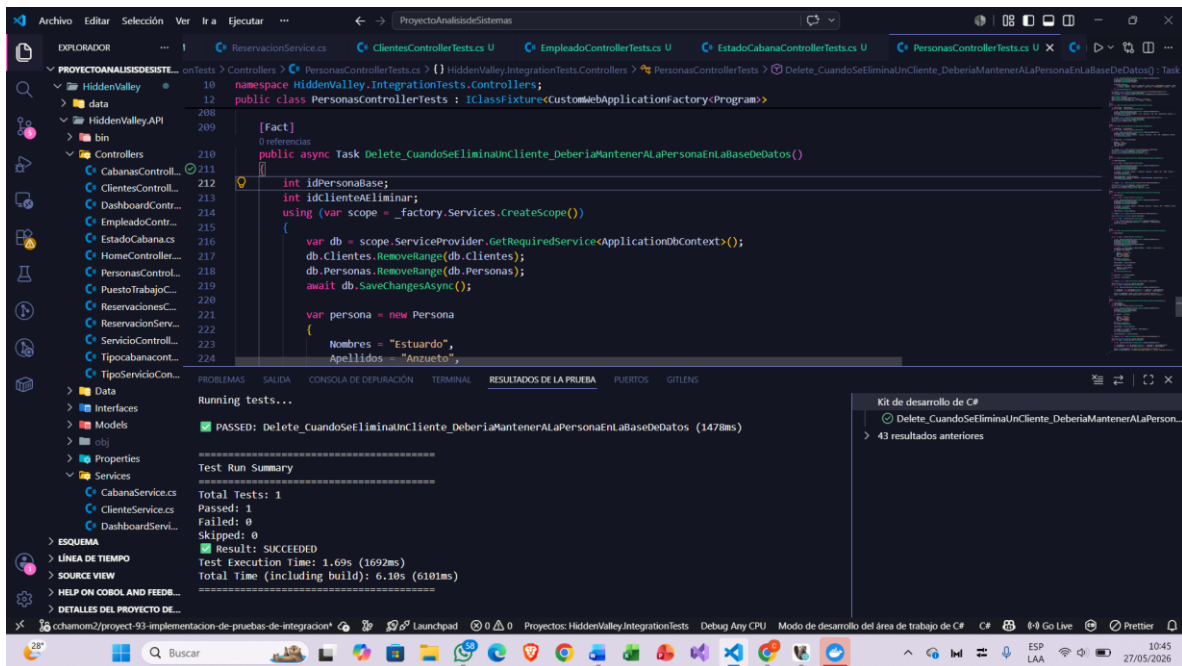
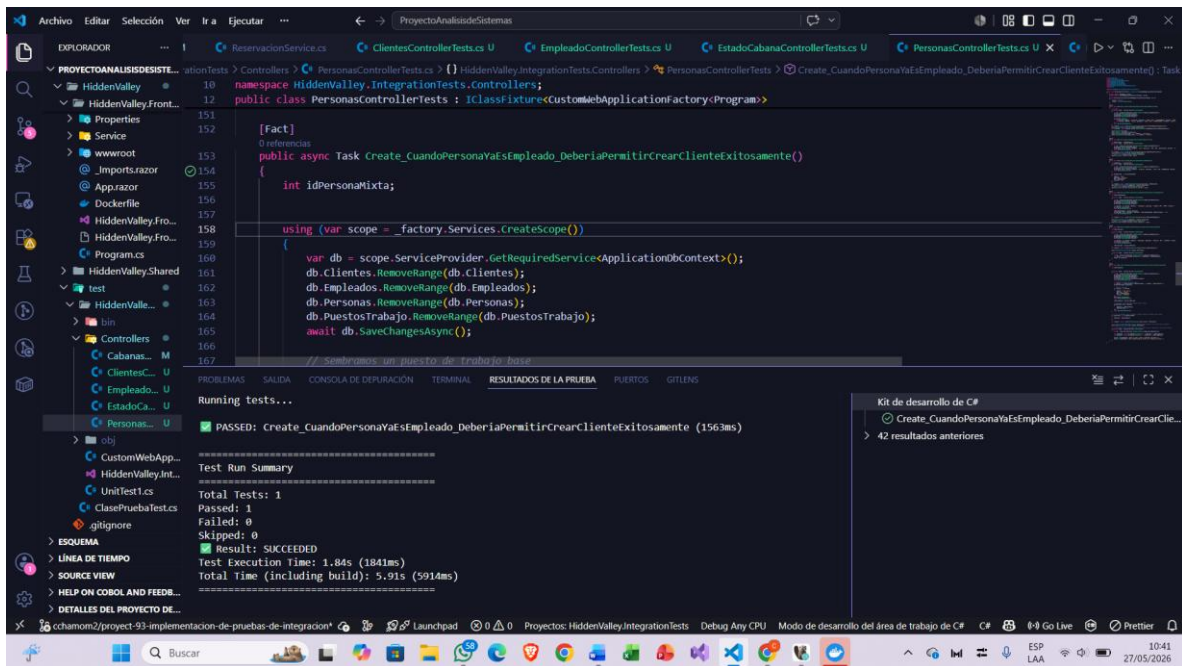




PERSONA







PUESTO TRABAJO

The screenshot shows the Visual Studio IDE with the 'PuestoTrabajoControllerTests.cs' file open. The code defines a test class `PuestoTrabajoControllerTests` that inherits from `IClassFixture<CustomWebApplicationFactory<Program>>`. A test method `GetAll_CuandoSearchEsNumerico_DeberiaFiltrarPorIdExacto` is shown, which uses `DbContext` to remove existing jobs and then adds two new jobs: 'Administrador' and 'Recepcionista'. The test passes, as indicated by the 'PASSED' status in the Test Results window.

```
namespace HiddenValley.IntegrationTests.Controllers;
public class PuestoTrabajoControllerTests : IClassFixture<CustomWebApplicationFactory<Program>>
{
    [Fact]
    public async Task GetAll_CuandoSearchEsNumerico_DeberiaFiltrarPorIdExacto()
    {
        using (var scope = _factory.Services.CreateScope())
        {
            var db = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();
            db.Empleados.RemoveRange(db.Empleados);
            db.PuestosTrabajo.RemoveRange(db.PuestosTrabajo);

            db.PuestosTrabajo.AddRange(
                new PuestoTrabajo { IdPuestoTrabajo = 1, Nombre = "Administrador", Descripcion = "Gerencia" },
                new PuestoTrabajo { IdPuestoTrabajo = 2, Nombre = "Recepcionista", Descripcion = "Atención" }
            );
            await db.SaveChangesAsync();
        }
    }
}
```

Test Run Summary

Test Name	Result
GetAll_CuandoSearchEsNumerico_DeberiaFiltrarPorIdExacto	Passed

Total Tests: 1
Passed: 1
Failed: 0
Skipped: 0
Result: SUCCEEDED
Test Execution Time: 1.51s (1511ms)
Total Time (including build): 4.99s (4989ms)

The screenshot shows the Visual Studio IDE with the 'PuestoTrabajoControllerTests.cs' file open. The code defines a test method `Create_CuandoNombreYaExiste_DeberiaRetornarBadRequest` which creates a new job with the name 'cocinero' and then attempts to create another job with the same name. The test passes, as indicated by the 'PASSED' status in the Test Results window.

```
namespace HiddenValley.IntegrationTests.Controllers;
public class PuestoTrabajoControllerTests : IClassFixture<CustomWebApplicationFactory<Program>>
{
    [Fact]
    public async Task Create_CuandoNombreYaExiste_DeberiaRetornarBadRequest()
    {
        using (var scope = _factory.Services.CreateScope())
        {
            var db = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();
            db.PuestosTrabajo.RemoveRange(db.PuestosTrabajo);
            db.PuestosTrabajo.Add(new PuestoTrabajo { Nombre = "cocinero", Descripcion = "Área Restaurante" });
            await db.SaveChangesAsync();
        }

        var dtoDuplicado = new PuestoTrabajoCreateDto { Nombre = "cocinero", Descripcion = "Intento duplicado" };

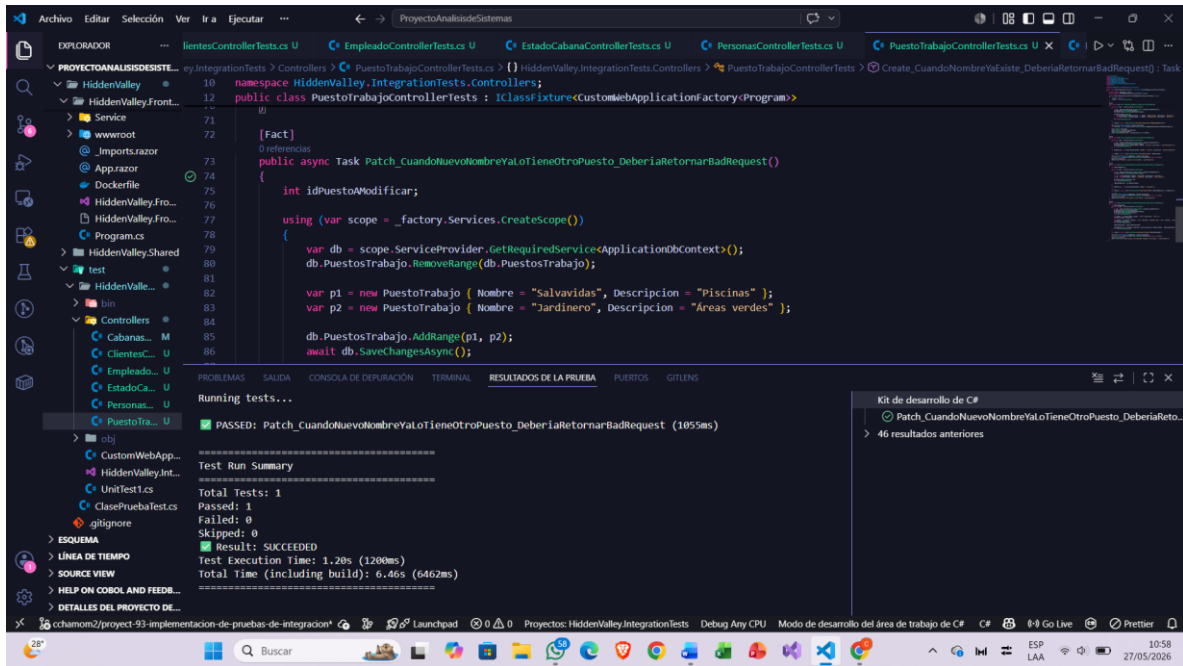
        var response = await _client.PostAsJsonAsync("api/puestotrabajo", dtoDuplicado);

        // Assert
    }
}
```

Test Run Summary

Test Name	Result
Create_CuandoNombreYaExiste_DeberiaRetornarBadRequest	Passed

Total Tests: 1
Passed: 1
Failed: 0
Skipped: 0
Result: SUCCEEDED
Test Execution Time: 1.51s (1506ms)
Total Time (including build): 4.95s (4947ms)



RESERVACIONES

```
namespace HiddenValley.IntegrationTests.Controllers;
public class ReservacionesControllerTests : IClassFixture<CustomWebApplicationFactory<Program>>
{
    private CustomWebApplicationFactory<Program> _factory;
    private HttpClient _client;

    public ReservacionesControllerTests(CustomWebApplicationFactory<Program> factory)
    {
        _factory = factory;
        _client = factory.CreateClient();
    }

    [Fact]
    public async Task Create_CuandoExisteTraslapeDeFechas_DeberiaRetornarBadRequest()
    {
        int idClienteParaTest;
        int idCabanaValida;
        string telefonoClienteA = "10101010";
        string telefonoClienteB = "55554444";

        using (var scope = _factory.Services.CreateScope())
        {
            var db = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();
            db.RegistroReservacion.RemoveRange(db.RegistroReservacion);
        }
    }
}
```

Running tests...
PASSED: Create_CuandoExisteTraslapeDeFechas_DeberiaRetornarBadRequest (1656ms)

Test Run Summary
Total Tests: 1
Passed: 1
Failed: 0
Skipped: 0
Result: SUCCEEDED
Test Execution Time: 1.79s (1792ms)
Total Time (including build): 5.11s (5113ms)

```
namespace HiddenValley.IntegrationTests.Controllers;
public class ReservacionesControllerTests : IClassFixture<CustomWebApplicationFactory<Program>>
{
    private CustomWebApplicationFactory<Program> _factory;
    private HttpClient _client;

    public ReservacionesControllerTests(CustomWebApplicationFactory<Program> factory)
    {
        _factory = factory;
        _client = factory.CreateClient();
    }

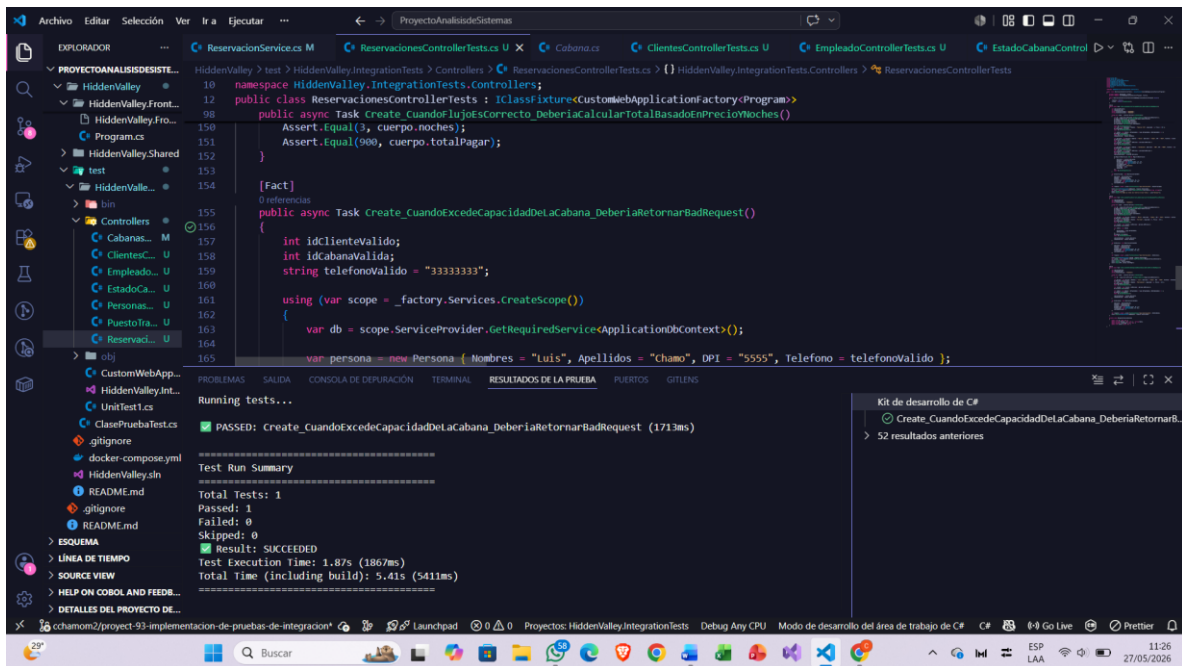
    [Fact]
    public async Task Create_CuandoFlujoEsCorrecto_DeberiaCalcularTotalBasadoEnPrecioMoches()
    {
        int idClienteValido;
        int idCabanaValida;
        string telefonoValido = "44444444";

        using (var scope = _factory.Services.CreateScope())
        {
            var db = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();
            db.RegistroReservacion.RemoveRange(db.RegistroReservacion);
            db.Cabanas.RemoveRange(db.Cabanas);
            db.Clientes.RemoveRange(db.Clientes);
            db.Personas.RemoveRange(db.Personas);
            db.TiposCabana.RemoveRange(db.TiposCabana);
            await db.SaveChangesAsync();

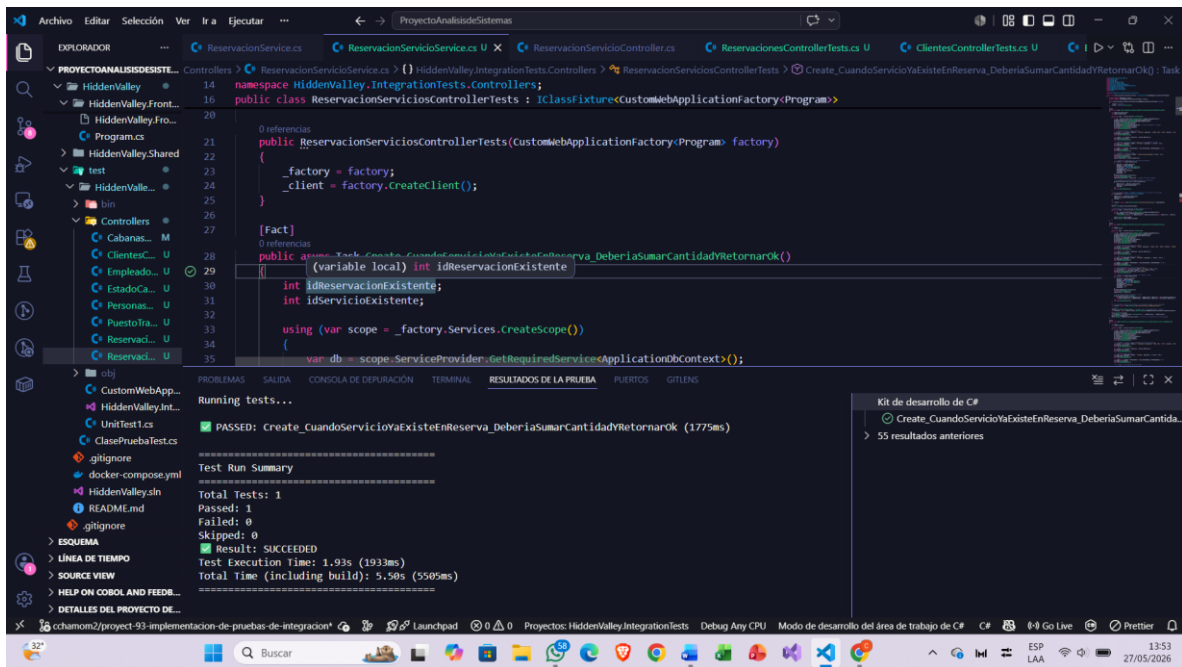
            var persona = new Persona { Nombre = "Maria", Apellidos = "Chavez", DPT = "4444", Telefono = telefonoValido };
        }
    }
}
```

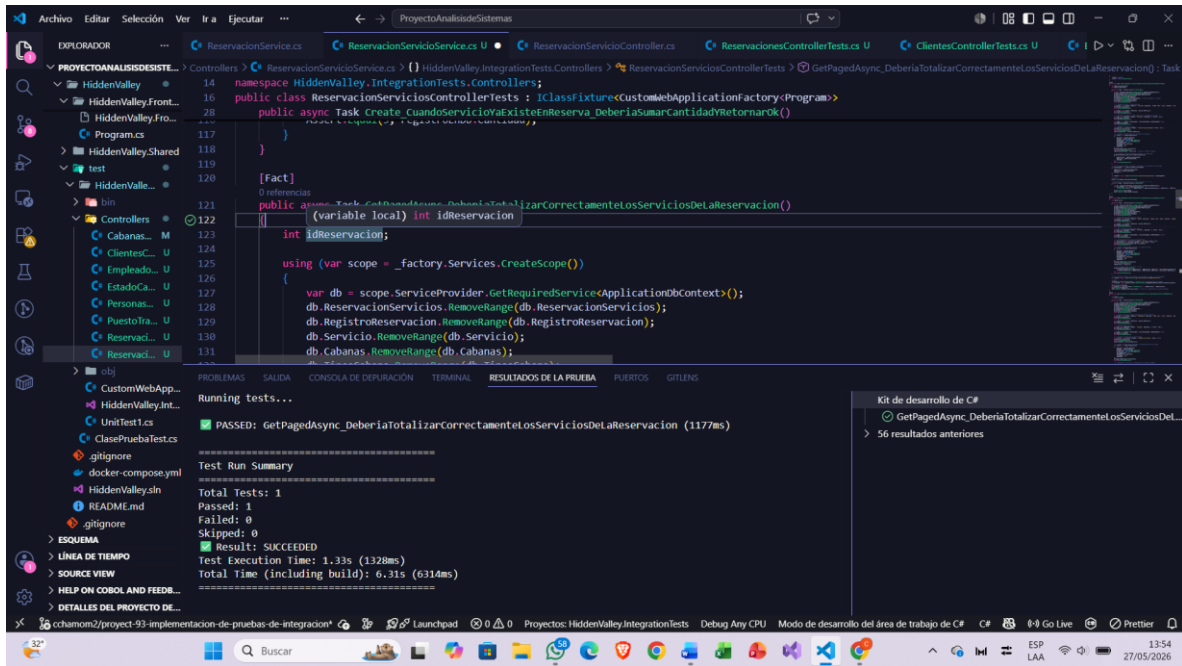
Running tests...
PASSED: Create_CuandoFlujoEsCorrecto_DeberiaCalcularTotalBasadoEnPrecioMoches (1263ms)

Test Run Summary
Total Tests: 1
Passed: 1
Failed: 0
Skipped: 0
Result: SUCCEEDED
Test Execution Time: 1.42s (1419ms)
Total Time (including build): 6.28s (6281ms)

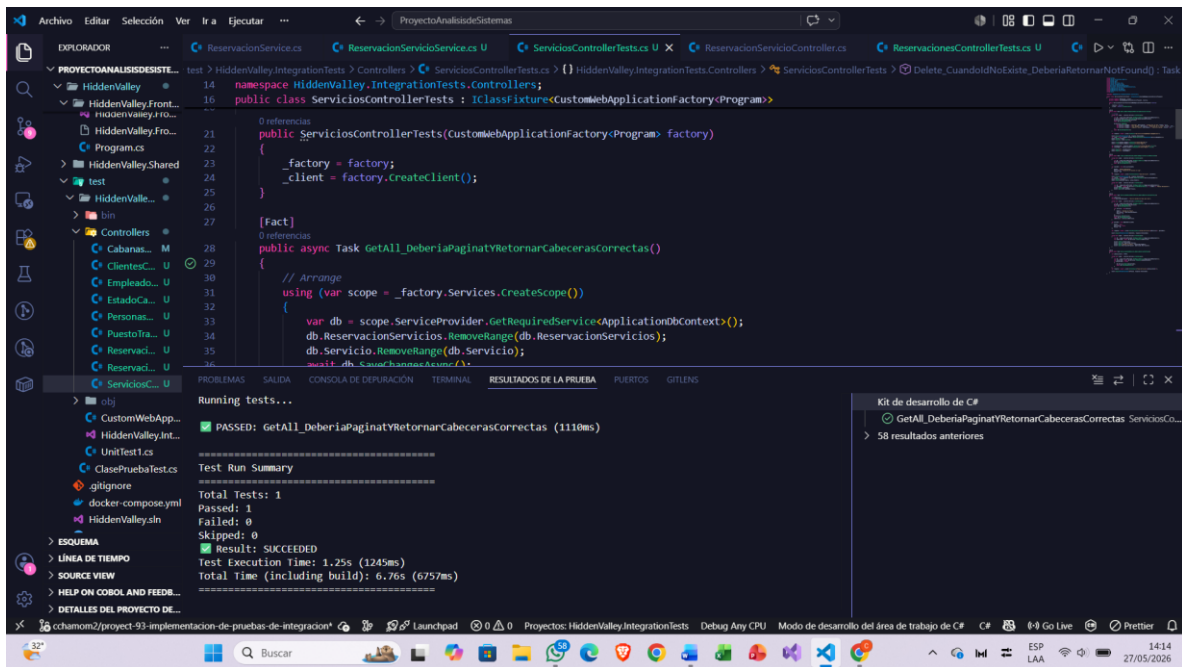


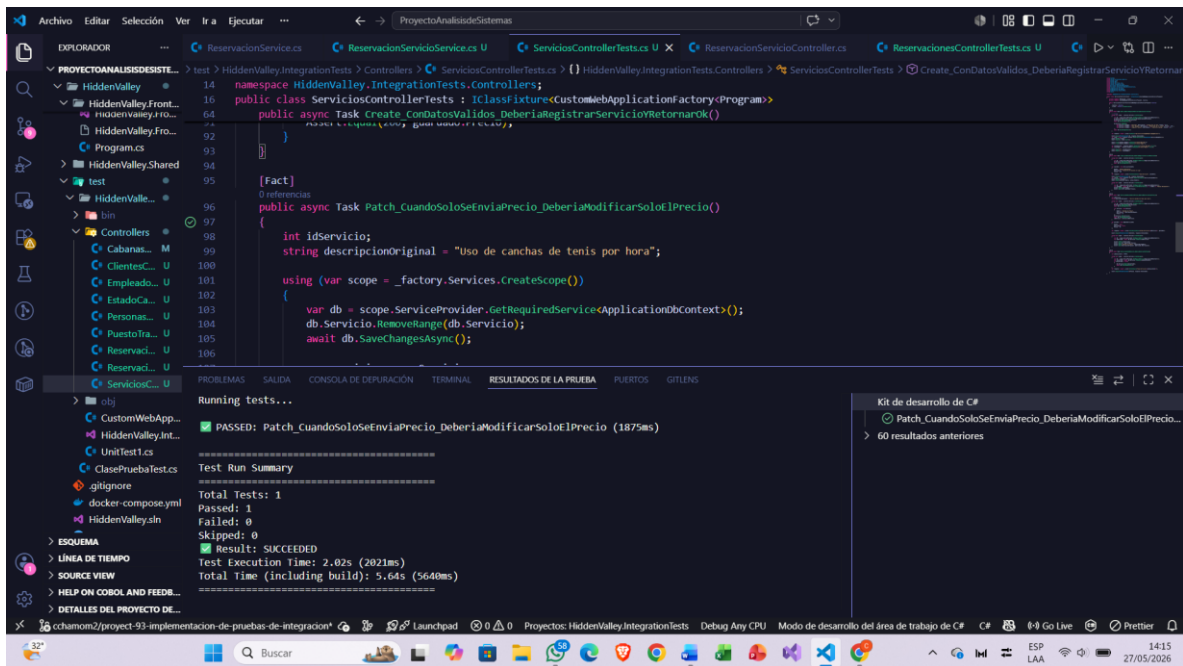
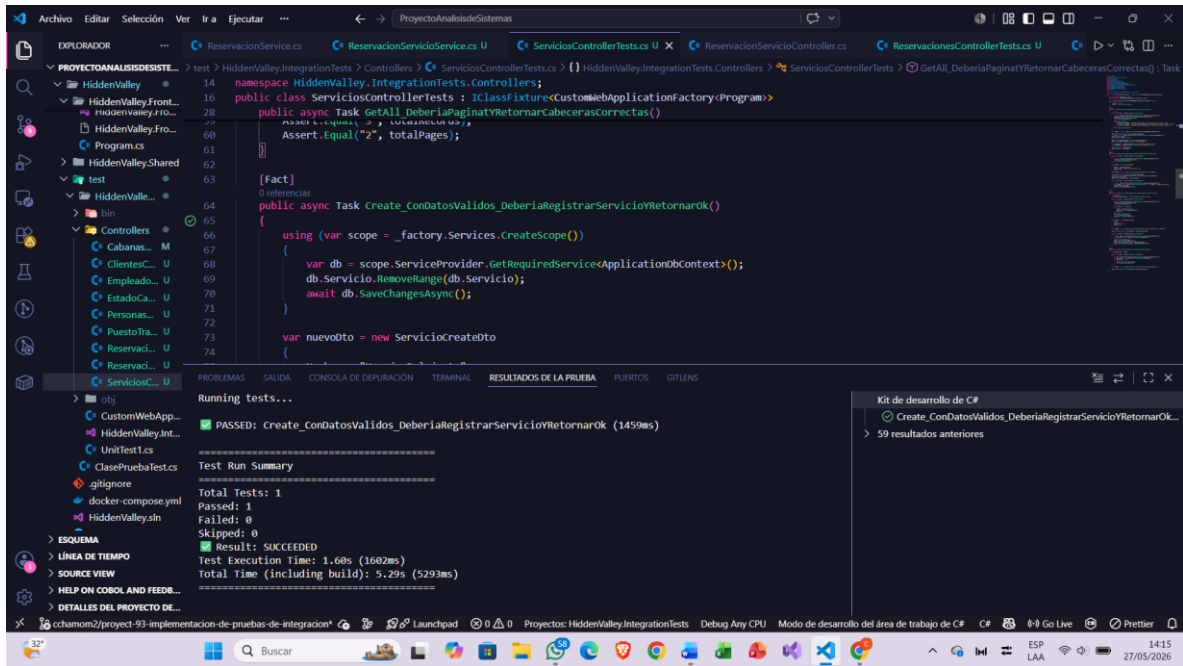
RESERVACION DE SERVICIOS





SERVICIO





TIPO CABAÑA

